

TREE 2

PROBLEM 5

QUESTION:

DS	Session	Tree 2	Question Information	Level 1	Challenge 75
Question description A beautiful <i>code</i> of a tree of n nodes is a sequence of $n-2$ integers that uniquely specifies the structure of the tree. The code is constructed as follows: As long as there are at least three nodes left, find a leaf with the smallest label, add the label of its only neighbour to the code, and remove the leaf from the tree. Given a beautiful code of a tree, your task is to construct the original tree. Constraints $3 \leq n \leq 2 \cdot 10^5$ $1 \leq a, b \leq n$ Input The first input line contains an integer n: the number of nodes. The nodes are numbered $1, 2, \dots, n$. The second line contains $n-2$ integers: the beautiful code. Output Print $n-1$ lines describing the edges of the tree. Each line has to contain two integers a and b: there is an edge between nodes a and b. You can print the edges in any order.					
✓ Logical Test Cases Test Case 1 INPUT (STDIN) 5 2 2 4 EXPECTED OUTPUT 1 2 3 2 2 4 4 5 Test Case 2 INPUT (STDIN) 5 3 1 2 EXPECTED OUTPUT 4 3 3 1 1 2 2 5					
✓ Mandatory Test Cases Test Case 1 KEYWORD <code>q.pop();</code>					
✓ Complexity Test Cases Test Case 1 CYCLOMATIC COMPLEXITY 3 Test Case 2 TOKEN COUNT 300 Test Case 3 NLOC 34					

SOURCE CODE:

```
#include <stdio.h>

int prufer_seq[200005];
int node_degrees[200005];

int main() {
    int total_nodes;
    if (scanf("%d", &total_nodes) != 1) return 0;

    for (int i = 1; i <= total_nodes; i++) node_degrees[i] = 1;
    for (int i = 0; i < total_nodes - 2; i++) {
        scanf("%d", &prufer_seq[i]);
        node_degrees[prufer_seq[i]]++;
    }

    int search_ptr = 1;
    while (node_degrees[search_ptr] != 1) search_ptr++;
    int current_leaf = search_ptr;

    for (int i = 0; i < total_nodes - 2; i++) {
        int parent_node = prufer_seq[i];
        printf("%d %d\n", current_leaf, parent_node);
        node_degrees[current_leaf]--;
        node_degrees[parent_node]--;

        if (node_degrees[parent_node] == 1 && parent_node < search_ptr) {
            current_leaf = parent_node;
        } else {
            search_ptr++;
            while (search_ptr <= total_nodes && node_degrees[search_ptr] != 1) {
                search_ptr++;
            }
            current_leaf = search_ptr;
        }
    }

    int final_u = 0, final_v = 0;
    for (int i = 1; i <= total_nodes; i++) {
        if (node_degrees[i] == 1) {
            if (final_u == 0) final_u = i;
            else final_v = i;
        }
    }
    printf("%d %d\n", final_u, final_v);
    return 0;
}
```

OUTPUT:

```
5
2 2 4
1 2
3 2
2 4
4 5
```

```
5
3 1 2
4 3
3 1
1 2
2 5
```